МОСКОВСКИЙ АВИАЦИОННЫЙ ИНСТИТУТ
(НАЦИОНАЛЬНЫЙ ИССЛЕДОВАТЕЛЬСКИЙ УНИВЕРСИТЕТ)

Институт №8 «Компьютерные науки и прикладная математика»
Кафедра 806 «Вычислительная математика и программирование»

Лабораторная работа №2

по курсу «Компьютерная графика»

Камера и освещение

Объект: правильный октаэдр

Выполнил: А. А. Устинов
Группа: М8О-307БВ-24

Преподаватель: В. Д. Бахарев

Москва, 2026

1. Условие

Цель работы — реализовать виртуальную камеру с возможностью её перемещения и освещение по модели Блинна-Фонга с использованием нескольких типов источников света.

Объект — правильный октаэдр из первой лабораторной работы. Программа работает в реальном времени с использованием Vulkan, GLFW и ImGui. Через пользовательский интерфейс изменяются параметры камеры (позиция, поворот, угол обзора), источников света (позиция, направление, цвет, интенсивность и параметры, свойственные типу источника), рассеянного света и материалов (альбедо, цвет блика, коэффициент блеска). Нормали вершин генерируются процедурно, модель Блинна-Фонга вычисляется во фрагментном шейдере, для точечных источников и прожекторов реализовано затухание по закону обратных квадратов.

Выполнено базовое задание — отрисовка объекта относительно камеры с одним направленным источником света — и три дополнительных задания:

1. внешние данные разделены на два набора дескрипторов: набор сцены (uniform-буфер с общими параметрами и storage-буферы с массивами источников) и набор модели (матрица преобразования и параметры материала);
2. массив точечных источников в storage-буфере с затуханием по закону обратных квадратов, добавлением и удалением источников в интерфейсе;
3. массив прожекторов в storage-буфере с затуханием, мягкими краями освещаемой области, добавлением и удалением источников в интерфейсе.

Для наглядности в сцену добавлен пол — плоскость под октаэдром, на которой видны пятна света от точечных источников и прожекторов.

2. Метод решения

2.1. Камера

Положение камеры задаётся точкой **e**, а направление взгляда — двумя углами: ψ — поворот вокруг вертикальной оси (рысканье, `yaw`) и φ — наклон вверх или вниз (тангаж, `pitch`). Единичный вектор направления взгляда получается из этих углов как точка на единичной сфере:

$$\mathbf{d} = \big(\cos\varphi \sin\psi,\ \ \sin\varphi,\ \ -\cos\varphi\cos\psi\big). \tag{1}$$

При $\psi = \varphi = 0$ получается $\mathbf{d} = (0, 0, -1)$: камера смотрит вдоль отрицательного направления оси Z, как принято в правой системе координат. Положительное ψ поворачивает взгляд вправо, положительное φ — вверх.

Матрица вида строится тем же способом, что и в первой работе, по положению камеры, точке наблюдения $\mathbf{e} + \mathbf{d}$ и направлению «вверх» $(0, 1, 0)$, а матрица перспективной проекции P — по углу обзора и отношению сторон окна. Их произведение $P \cdot V$ передаётся в шейдер как одна матрица.

Угол наклона ограничен диапазоном $\pm 89°$. При $\varphi = \pm 90°$ направление взгляда совпало бы с направлением «вверх», их векторное произведение обратилось бы в ноль, и базис камеры стал бы неопределённым.

2.2. Нормали

Для расчёта освещения в каждой вершине помимо позиции хранится нормаль — единичный вектор, перпендикулярный поверхности. Нормали генерируются процедурно: для треугольника с вершинами **a**, **b**, **c**, перечисленными против часовой стрелки при взгляде снаружи,

$$\mathbf{n} = \frac{(\mathbf{b} - \mathbf{a}) \times (\mathbf{c} - \mathbf{a})}{\big\|(\mathbf{b} - \mathbf{a}) \times (\mathbf{c} - \mathbf{a})\big\|}. \tag{2}$$

Векторное произведение двух сторон треугольника перпендикулярно его плоскости, а порядок обхода вершин определяет, что нормаль направлена наружу.

В первой работе октаэдр хранился шестью вершинами: каждая вершина использовалась четырьмя гранями. Для освещения так нельзя: в каждой вершине октаэдра сходятся четыре грани с разными нормалями, а атрибут вершины может хранить только одну. Если усреднить их, освещённость плавно перетекала бы с грани на грань, и октаэдр выглядел бы скруглённым. Поэтому каждая грань получает свои три вершины с одинаковой нормалью: 24 вершины для октаэдра и 6 для пола. Такие нормали называются плоскими.

При переводе в мировые координаты нормаль умножается не на матрицу модели M, а на обратную транспонированную к её верхнему левому блоку 3×3:

$$\mathbf{n}_{\text{мир}} = \left(M_{3\times 3}^{-1}\right)^{\mathsf{T}} \mathbf{n}. \tag{3}$$

При неравномерном масштабировании обычная матрица модели искажает нормаль, и она перестаёт быть перпендикулярной поверхности; обратная транспонированная матрица этот эффект компенсирует. В данной работе модели только переносятся, и результат совпал бы, но формула записана в общем виде.

2.3. Модель освещения Блинна-Фонга

Обозначения, используемые в формулах освещения, сведены в таблицу 1.

Цвет точки складывается из рассеянного света и вкладов всех источников:

$$\mathbf{c} = \mathbf{A} \odot \mathbf{k}_d + \sum_i \Big(\mathbf{k}_d \max(\mathbf{n} \cdot \mathbf{l}_i,\ 0) + \mathbf{k}_s \max(\mathbf{n} \cdot \mathbf{h}_i,\ 0)^p \Big) \odot \mathbf{E}_i, \tag{4}$$

где полувектор

$$\mathbf{h}_i = \frac{\mathbf{l}_i + \mathbf{v}}{\|\mathbf{l}_i + \mathbf{v}\|}. \tag{5}$$

Первое слагаемое в скобках — диффузное отражение, закон Ламберта: освещённость пропорциональна косинусу угла между нормалью и направлением на источник. Поверхность, обращённая к источнику, освещена сильнее

Таблица 1 – Обозначения в формулах освещения

Обозначение	Что обозначает
$\mathbf{n}$	единичная нормаль к поверхности в освещаемой точке
$\mathbf{v}$	единичный вектор от точки к камере
$\mathbf{l}_i$	единичный вектор от точки к i-му источнику
$\mathbf{h}_i$	полувектор — направление посередине между $\mathbf{l}_i$ и $\mathbf{v}$
$\mathbf{k}_d$	альбедо — цвет рассеянного (диффузного) отражения материала
$\mathbf{k}_s$	цвет блика материала
p	коэффициент блеска: чем больше, тем меньше и резче блик
$\mathbf{A}$	рассеянный (фоновый) свет: цвет, умноженный на интенсивность
$\mathbf{E}_i$	свет, приходящий от i-го источника в точку
$\mathbf{C}_i,\ I_i$	цвет и интенсивность i-го источника
r_i	расстояние от точки до i-го источника
$\odot$	покомпонентное умножение: каналы R, G и B перемножаются по отдельности

всего, повёрнутая боком — не освещена вовсе. Второе слагаемое — блик: он ярок, когда полувектор $\mathbf{h}_i$ близок к нормали, то есть когда камера находится рядом с направлением зеркального отражения луча. Показатель p определяет, насколько быстро блик гаснет при отклонении от этого направления.

Отличие от исходной модели Фонга — в способе оценки блика. В модели Фонга вычисляется отражённый луч и его угол с направлением на камеру. Полувектор вычисляется проще и ведёт себя лучше при скользящих углах падения: угол между отражённым лучом и направлением на камеру может превысить $90°$, и блик резко обрывается, тогда как угол между $\mathbf{h}_i$ и нормалью остаётся в допустимых пределах.

Если поверхность повёрнута от источника ($\mathbf{n} \cdot \mathbf{l}_i \leq 0$), его вклад, включая блик, принимается равным нулю: иначе на неосвещённой стороне появ-

лялись бы паразитные блики.

2.4. Типы источников света

Типы источников различаются только тем, как вычисляются направление $\mathbf{l}_i$ и приходящий свет $\mathbf{E}_i$.

Направленный источник моделирует очень удалённый источник вроде солнца: все лучи параллельны, направление $\mathbf{l}$ одинаково для всех точек сцены, затухания нет:

$$\mathbf{E} = \mathbf{C} \cdot I. \tag{6}$$

Точечный источник излучает одинаково во все стороны из одной точки. Направление $\mathbf{l}_i$ своё для каждой точки поверхности, а освещённость убывает по закону обратных квадратов:

$$\mathbf{E}_i = \frac{\mathbf{C}_i \cdot I_i}{r_i^2}. \tag{7}$$

Физический смысл закона: один и тот же световой поток на расстоянии r распределяется по поверхности сферы площадью $4\pi r^2$, поэтому на единицу площади приходится доля, обратно пропорциональная r^2. При $r \to 0$ формула даёт бесконечность, поэтому в шейдере квадрат расстояния ограничен снизу малым числом.

Прожектор — точечный источник, свет которого ограничен конусом с осью вдоль направления $\mathbf{s}_i$. Для мягкого края задаются два угла, отсчитываемых от оси: внутренний θ_{in}, внутри которого яркость полная, и внешний θ_{out}, за которым свет пропадает. Между ними яркость плавно спадает:

$$\mathbf{E}_i = \frac{\mathbf{C}_i \cdot I_i}{r_i^2} \cdot k_{\text{край}}, \qquad k_{\text{край}} = \operatorname{clamp}\left(\frac{\cos\theta - \cos\theta_{\text{out}}}{\cos\theta_{\text{in}} - \cos\theta_{\text{out}}},\ 0,\ 1 \right), \tag{8}$$

где θ — угол между осью прожектора и направлением от прожектора на освещаемую точку, $\cos\theta = -\mathbf{l}_i \cdot \mathbf{s}_i$, а функция $\operatorname{clamp}(x, 0, 1)$ ограничивает значение отрезком $[0, 1]$.

Внутри внутреннего конуса числитель больше знаменателя и множитель равен единице; за внешним конусом числитель отрицателен и множитель равен нулю; между конусами он линейно меняется от единицы до нуля. Сравнение ведётся по косинусам, а не по самим углам: на отрезке от 0° до 90° косинус монотонно убывает, поэтому сравнение косинусов эквивалентно сравнению углов, а вычислять арккосинус для каждого пикселя не требуется. Косинусы обоих углов вычисляются один раз на процессоре.

Если внутренний и внешний углы равны, знаменатель обращается в ноль; в шейдере он ограничен снизу малым числом, и край пятна становится резким.

2.5. Используемые объекты Vulkan

В отличие от первой работы, где данные передавались в шейдер через push-константы, здесь используются наборы дескрипторов. Внешние данные разделены на два набора (дополнительное задание 1).

Набор 0 — данные сцены, общие для всех объектов:

- привязка 0 — uniform-буфер: матрица $P \cdot V$, положение камеры, рассеянный свет, параметры направленного источника и количество точечных источников и прожекторов;
- привязка 1 — storage-буфер с массивом точечных источников;
- привязка 2 — storage-буфер с массивом прожекторов.

Набор 1 — данные модели: uniform-буфер с матрицей преобразования и параметрами материала. У каждой модели свой буфер и свой набор.

При отрисовке набор сцены привязывается один раз за кадр, а набор модели — перед каждым объектом. Если объектов много, меняется только небольшой набор с данными модели, а данные сцены остаются привязанными.

Создаются следующие объекты Vulkan:

- `VkDescriptorSetLayout` — две раскладки наборов, описывающие, какие ресурсы на каких привязках ожидают шейдеры;
- `VkDescriptorPool` — пул, из которого выделяются наборы: один набор сцены и по одному на модель;

- `VkDescriptorSet` — сами наборы, связанные с буферами один раз при запуске, поскольку буферы не пересоздаются;
- `VkBuffer` — uniform-буфер сцены, два storage-буфера с источниками, а для каждой модели — вершинный буфер и uniform-буфер. Все буферы размещены в памяти, доступной процессору, и постоянно отображены в его адресное пространство, поэтому данные переписываются каждый кадр простым копированием;
- `VkPipelineLayout` — раскладка конвейера с двумя наборами дескрипторов;
- `VkPipeline` — графический конвейер. Формат вершины — позиция и нормаль, по три числа одинарной точности; остальные настройки (тест глубины, отсечение нелицевых граней) совпадают с первой работой.

Storage-буферы используются для массивов источников, потому что только в них массив может быть объявлен в шейдере без указания размера:

```
layout(std430, set = 0, binding = 1) readonly buffer PointLights {
    PointLight point_lights[];
};
```

В uniform-буфере размер массива обязан быть константой, известной при компиляции шейдера. Количество действующих источников передаётся в uniform-буфере сцены, и цикл в шейдере проходит ровно столько элементов. На стороне приложения буфер выделяется один раз с запасом на 64 источника каждого типа; это ограничение размера буфера, а не шейдера, и меняется изменением одной константы.

Раскладка данных. Расположение полей в буферах должно в точности совпадать с тем, как их читает шейдер. Для uniform-буферов действуют правила выравнивания `std140`, для storage-буферов — `std430`; при обоих правилах трёхкомпонентный вектор занимает место четырёхкомпонентного. Поэтому все векторы хранятся как `vec4`, а свободный четвёртый компонент используется для смежной величины: интенсивности у цвета источника, коэффициента блеска у цвета блика.

Синхронизация. Буферы переписываются при записи команд кадра, а не при обработке интерфейса. К этому моменту стартовый код дожидается завершения предыдущего кадра, поэтому видеокарта гарантированно не читает буферы в момент записи.

2.6. Интерфейс

Окно интерфейса разделено на разделы: камера, рассеянный свет, направленный источник, точечные источники, прожекторы и материалы. В разделах источников каждый элемент списка имеет собственные параметры и кнопку удаления, а под списком находится кнопка добавления нового источника (дополнительные задания 2 и 3). Для прожектора внешний угол не может быть сделан меньше внутреннего. В разделе материалов для каждой модели настраиваются альбедо, цвет блика и коэффициент блеска.

Ниже приведён фрагментный шейдер, в котором реализована модель освещения.

```
#version 450

layout(location = 0) in vec3 world_position;
layout(location = 1) in vec3 world_normal;

layout(location = 0) out vec4 out_color;

layout(set = 0, binding = 0) uniform SceneData {
    mat4 view_projection;
    vec4 camera_position;
    vec4 ambient;
    vec4 light_direction;
    vec4 light_color;
    uint point_light_count;
    uint spot_light_count;
} scene;

struct PointLight {
    vec4 position;          // xyz
    vec4 color;             // rgb — цвет, a — интенсивность
};

struct SpotLight {
    vec4 position;          // xyz
```

```
    vec4 direction;         // xyz — куда светит
    vec4 color;             // rgb — цвет, a — интенсивность
    vec4 cutoff;            // x — косинус внутреннего угла, y — внешнего
};

// Массивы без указания размера: сколько элементов, знает только буфер
// (доп. задания 2 и 3)
layout(std430, set = 0, binding = 1) readonly buffer PointLights {
    PointLight point_lights[];
};

layout(std430, set = 0, binding = 2) readonly buffer SpotLights {
    SpotLight spot_lights[];
};

layout(set = 1, binding = 0) uniform ModelData {
    mat4 transform;
    vec4 albedo;
    vec4 specular;
} object;

// Модель Блинна-Фонга для одного источника.
// N — нормаль, V — направление на камеру, L — направление на источник,
// radiance — цвет источника с учётом интенсивности и затухания.
vec3 blinnPhong(vec3 N, vec3 V, vec3 L, vec3 radiance) {
    float n_dot_l = max(dot(N, L), 0.0);
    if (n_dot_l == 0.0) {
        return vec3(0.0);
    }

    vec3 H = normalize(L + V);
    float n_dot_h = max(dot(N, H), 0.0);

    vec3 diffuse = object.albedo.rgb * n_dot_l;
    vec3 specular = object.specular.rgb * pow(n_dot_h, object.specular.a);

    return (diffuse + specular) * radiance;
}

// Выполняется для каждого пикселя: складывает вклад всех источников света.
void main() {
    vec3 N = normalize(world_normal);
    vec3 V = normalize(scene.camera_position.xyz - world_position);

    // Рассеянный свет
    vec3 color = scene.ambient.rgb * object.albedo.rgb;
```

```
71
72      // Направленный источник: лучи параллельны, затухания нет
73      vec3 sun_direction = normalize(-scene.light_direction.xyz);
74      color += blinnPhong(N, V, sun_direction, scene.light_color.rgb * scene.light_color.a);
75
76      // Доп. задание 2: точечные источники, затухание по закону обратных квадратов
77      for (uint i = 0; i < scene.point_light_count; ++i) {
78          PointLight light = point_lights[i];
79
80          vec3 to_light = light.position.xyz - world_position;
81          float distance_squared = max(dot(to_light, to_light), 0.0001);
82          vec3 L = to_light / sqrt(distance_squared);
83
84          vec3 radiance = light.color.rgb * light.color.a / distance_squared;
85          color += blinnPhong(N, V, L, radiance);
86      }
87
88      // Доп. задание 3: прожекторы — затухание и мягкий край пятна
89      for (uint i = 0; i < scene.spot_light_count; ++i) {
90          SpotLight light = spot_lights[i];
91
92          vec3 to_light = light.position.xyz - world_position;
93          float distance_squared = max(dot(to_light, to_light), 0.0001);
94          vec3 L = to_light / sqrt(distance_squared);
95
96          // Косинус угла между осью прожектора и направлением на точку.
97          // Внутри внутреннего конуса — 1, за внешним — 0, между ними плавный переход
98          float cos_angle = dot(-L, normalize(light.direction.xyz));
99          float cos_inner = light.cutoff.x;
100         float cos_outer = light.cutoff.y;
101         float edge = clamp((cos_angle - cos_outer) / max(cos_inner - cos_outer, 0.0001),
102                            0.0, 1.0);
103
104         vec3 radiance = light.color.rgb * light.color.a * edge / distance_squared;
105         color += blinnPhong(N, V, L, radiance);
106     }
107
108     out_color = vec4(color, 1.0);
109 }
```

Листинг 1: Фрагментный шейдер: модель Блинна-Фонга для трёх типов источников

3. Результаты

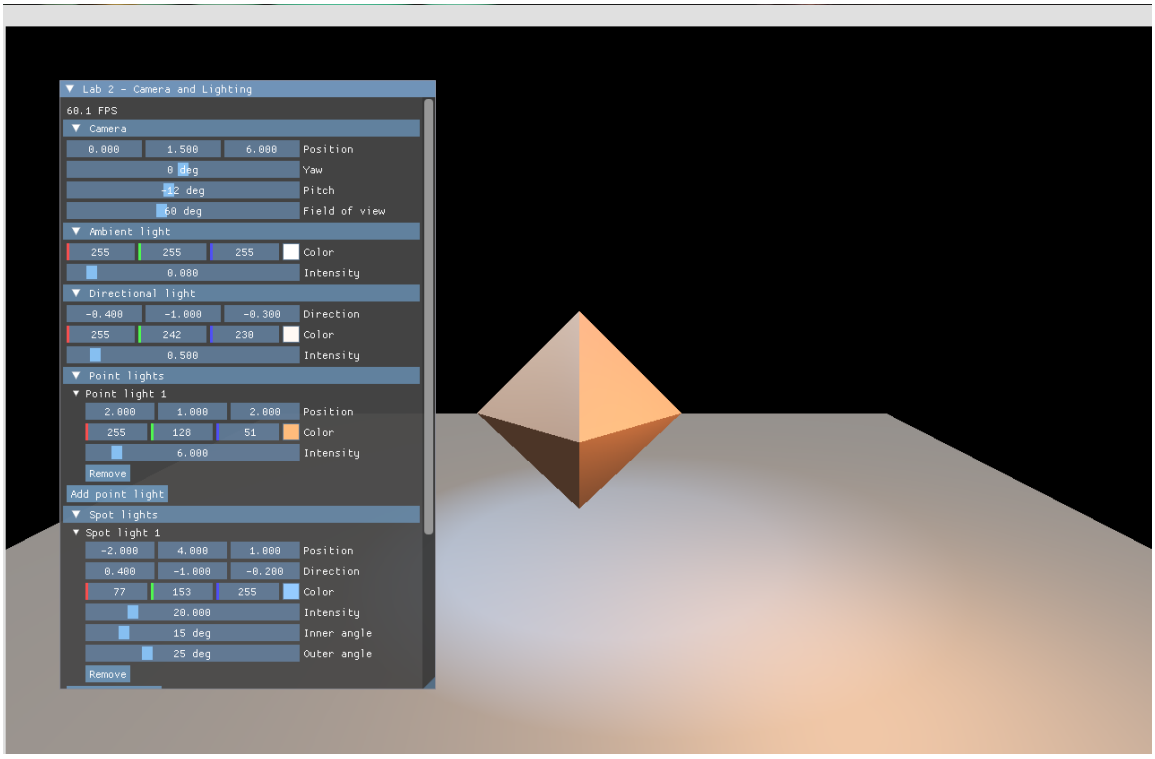


Рисунок 1 – Сцена при запуске: направленный источник, точечный источник оранжевого цвета и голубой прожектор. Верхние грани октаэдра освещены сверху, нижние темнее; на полу видны пятна от точечного источника и прожектора

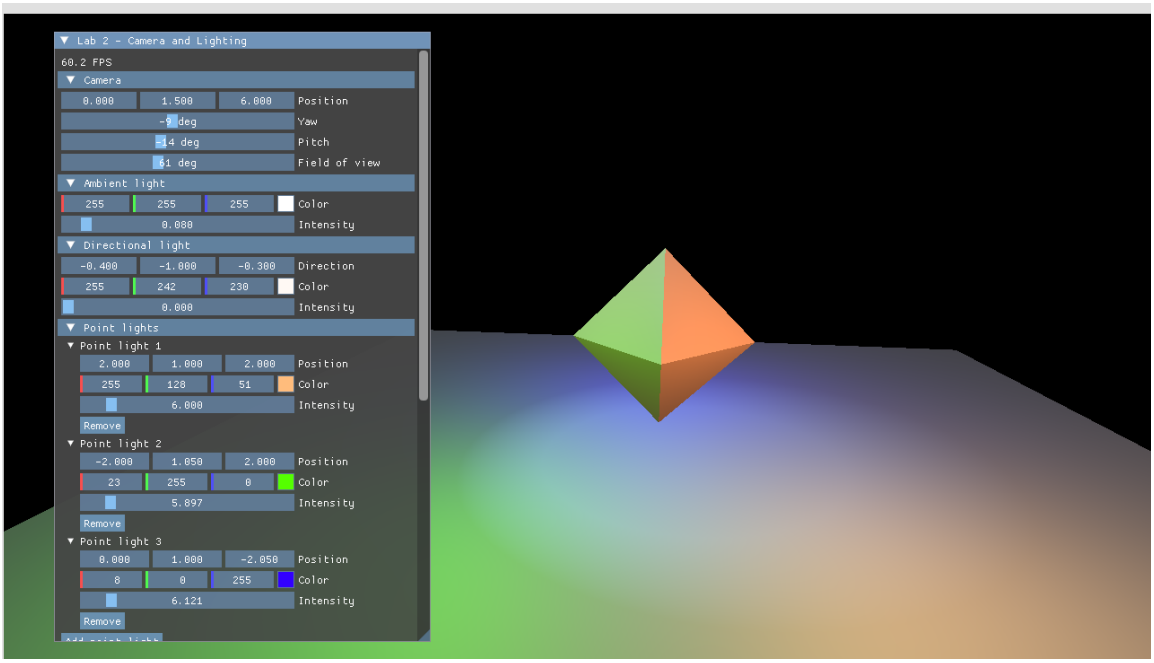


Рисунок 2 – Три точечных источника разных цветов при выключенном направленном свете. Пятна на полу гаснут с расстоянием по закону обратных квадратов, грани октаэдра окрашены ближайшими к ним источниками

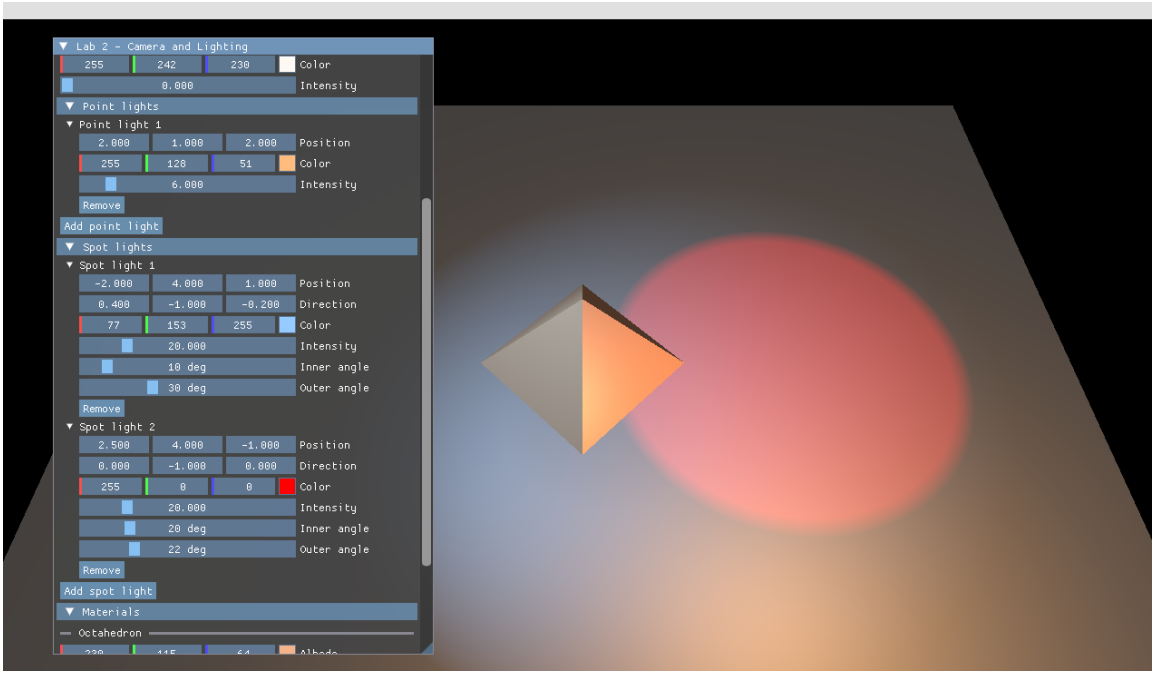


Рисунок 3 – Два прожектора с разными углами. Голубой с внутренним углом $10°$ и внешним $30°$ даёт широкий размытый край, красный с углами $20°$ и $22°$ — почти резкий

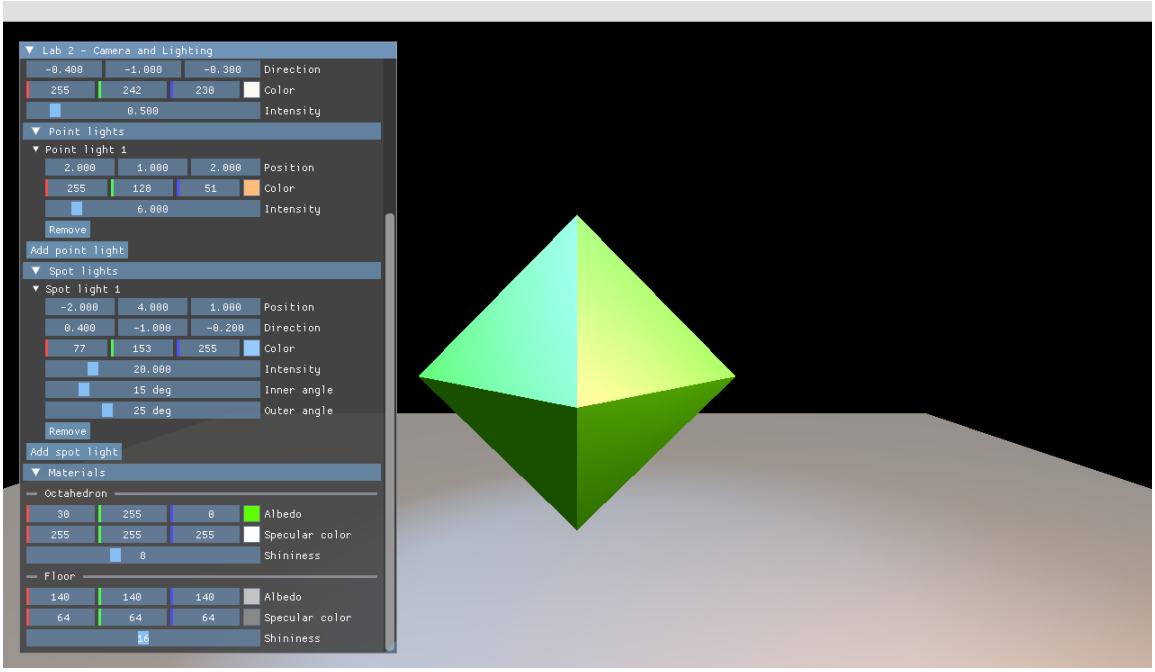


Рисунок 4 – Изменение материала: зелёное альбедо и коэффициент блеска 8. При малом коэффициенте блик широкий и растекается по грани; цвет блика задаётся отдельно от альбедо и остаётся светлым

4. Выводы

В ходе работы я научился управлять виртуальной камерой через положение и два угла поворота, рассчитывать освещение по модели Блинна-Фонга для трёх типов источников — направленного, точечного и прожектора — и передавать данные в шейдеры через наборы дескрипторов, uniform- и storage-буферы.

Главное, что я понял: освещение требует от геометрии больше, чем просто позиции вершин. Нормаль — свойство поверхности, а не вершины, и у октаэдра в одной вершине сходятся четыре грани с разными нормалями. Поэтому вершины, которые в первой работе были общими для граней, пришлось продублировать — по три на каждую грань, — иначе плоские грани освещались бы как скруглённые.

Типы источников различаются лишь тем, как вычисляются направление на источник и приходящий от него свет, а сама модель отражения для них общая. Мягкий край прожектора получается линейным переходом между косинусами двух углов, а сравнение по косинусам позволяет обойтись без

вычисления углов в каждом пикселе.

Разделение данных на набор сцены и набор модели оказалось удобным даже при двух объектах: данные сцены привязываются один раз за кадр, а для каждого объекта меняется только небольшой набор с его матрицей и материалом. Storage-буферы позволили объявить массивы источников в шейдере без фиксированного размера: количество источников меняется в интерфейсе без изменения шейдера.